

DBMS & SQL LAB (BCS552)

B.Tech CSE 5th SEMESTER

SUBMITTED TO:

MR. ROSHAN RAJ

SUBMITTED BY:

PRATIPAL SINGH

ROLL NO.: 2410303351

INDEX

DBMS & SQL LAB (BCS552) | B.Tech CSE 5th Semester

NO.	EXPERIMENT TITLE	PAGE
01	Creating a Table and Adding Rows	3
02	Updating and Deleting Rows	4
03	Selecting Columns and Rows with WHERE	5
04	Combining Conditions	6
05	Pattern Matching with LIKE	7
06	Sorting Rows and Removing Repeats	8
07	Using Aggregate Functions	9
08	Grouping Rows and Using HAVING	10
09	Aliases and CASE	11
10	Joining Two Tables with INNER JOIN	12
11	Other Kinds of Joins	14
12	Using a Subquery	16
13	UNION and INTERSECT	17
14	Creating and Using a View	19
15	Indexes and Adding a Row	20

APPENDIX: Actual SQL execution results (SQLite 3.46.1) | Experiments 01·15: pages 22·36

Page numbers indicate the starting page of each experiment.

EXPERIMENT 01

Creating a Table and Adding Rows

1. Aim: Create a table named student, insert three records in one INSERT statement and display all rows.

2. Software / Tools Required: SQL-supported DBMS (such as MySQL), database client/SQL editor and a computer.

3. Theory: CREATE TABLE defines a table and its columns. INSERT INTO adds records. SELECT retrieves data; the asterisk (*) selects every column.

4. Algorithm / Procedure:

• Open the SQL editor and connect to the database.

• Execute the setup statements, if provided, to create and populate the required tables.

• Run each task query in the given order.

• Verify the returned rows or the changes made to the table.

5. Program / Queries / Commands:

```
CREATE TABLE student (
roll_no INT,
name VARCHAR(30),
marks INT
);

INSERT INTO student (roll_no, name, marks)
VALUES (101, 'Asha', 78),
(102, 'Rohan', 64),
(103, 'Neha', 88);

SELECT * FROM student;
```

7. Input: (101, 'Asha', 78), (102, 'Rohan', 64), (103, 'Neha', 88)

8. Output:

roll_no	name	marks
101	Asha	78
102	Rohan	64
103	Neha	88

9. Result: The experiment was performed successfully. The required SQL operation was executed and the output was verified against the sample data.

10. Viva-Voce Questions with Answers:

Q. What is SQL?

Ans. SQL stands for Structured Query Language. It is used to manage data in relational databases.

Q. What is a table?

Ans. A table stores data in rows and columns.

Q. What does CREATE TABLE do?

Ans. It creates a new table with the specified columns and data types.

EXPERIMENT 02

Updating and Deleting Rows

1. Aim: Update stock quantities and delete a row, checking the table after each operation.

2. Software / Tools Required: SQL-supported DBMS (such as MySQL), database client/SQL editor and a computer.

3. Theory: UPDATE modifies existing records. DELETE removes selected records. WHERE identifies which rows should be affected; without WHERE, all rows may be changed or deleted.

4. Algorithm / Procedure:

• Open the SQL editor and connect to the database.

• Execute the setup statements, if provided, to create and populate the required tables.

• Run each task query in the given order.

• Verify the returned rows or the changes made to the table.

5. Setup / Initial Data:

```
CREATE TABLE stock (item_code INT, item_name VARCHAR(30), quantity INT);
INSERT INTO stock (item_code, item_name, quantity)
VALUES (1, 'Notebook', 50), (2, 'Pen', 120), (3, 'Bag', 0);
```

6. Program / Queries / Commands:

```
-- Task 1: Set Notebook quantity to 80
UPDATE stock SET quantity = 80 WHERE item_code = 1;
SELECT * FROM stock;

-- Task 2: Add 10 to Pen quantity
UPDATE stock SET quantity = quantity + 10 WHERE item_code = 2;
SELECT * FROM stock;

-- Task 3: Delete item with code 3
DELETE FROM stock WHERE item_code = 3;
SELECT * FROM stock;
```

7. Input: Initial stock: Notebook = 50, Pen = 120, Bag = 0.

8. Output:

item_code	item_name	quantity
1	Notebook	80
2	Pen	130

After Task 1: Notebook = 80, Pen = 120, Bag = 0. After Task 2: Notebook = 80, Pen = 130, Bag = 0. After Task 3: only Notebook and Pen remain.

9. Result: The experiment was performed successfully. The required SQL operation was executed and the output was verified against the sample data.

10. Viva-Voce Questions with Answers:

Q. What is the UPDATE command?

Ans. UPDATE changes values in existing rows of a table.

Q. Why is WHERE used with UPDATE?

Ans. It restricts the update to rows that satisfy the condition.

Q. What is the difference between DELETE and DROP?

Ans. DELETE removes rows from a table; DROP removes the database object such as the table itself.

EXPERIMENT 03

Selecting Columns and Rows with WHERE

1. Aim: Display selected columns and retrieve rows that satisfy WHERE conditions.

2. Software / Tools Required: SQL-supported DBMS (such as MySQL), database client/SQL editor and a computer.

3. Theory: SELECT chooses columns to display. WHERE filters rows according to a condition, such as a comparison between marks and a number.

4. Algorithm / Procedure:

• Open the SQL editor and connect to the database.

• Execute the setup statements, if provided, to create and populate the required tables.

• Run each task query in the given order.

• Verify the returned rows or the changes made to the table.

5. Setup / Initial Data:

```
CREATE TABLE student (roll_no INT, name VARCHAR(30), marks INT);
INSERT INTO student (roll_no, name, marks)
VALUES (101, 'Asha', 78), (102, 'Rohan', 64),
(103, 'Neha', 88), (104, 'Deepak', 52);
```

6. Program / Queries / Commands:

```
-- Task 1
SELECT name, marks FROM student;

-- Task 2
SELECT * FROM student WHERE marks > 60;

-- Task 3
SELECT name FROM student WHERE roll_no = 103;
```

7. Input: Four student records with roll numbers 101–104 and the marks given in the setup.

8. Output:

roll_no	name	marks
101	Asha	78
102	Rohan	64
103	Neha	88

Task 1 displays name and marks for all four students. Task 2 returns Asha (78), Rohan (64) and Neha (88). Task 3 returns Neha.

9. Result: The experiment was performed successfully. The required SQL operation was executed and the output was verified against the sample data.

10. Viva-Voce Questions with Answers:

Q. What is the purpose of SELECT?

Ans. SELECT retrieves data from one or more database tables.

Q. What does WHERE do?

Ans. WHERE filters rows based on a specified condition.

Q. What is the difference between SELECT * and SELECT name?

Ans. SELECT * returns all columns, while SELECT name returns only the name column.

EXPERIMENT 04

Combining Conditions

1. Aim: Use AND, BETWEEN and IN to filter records with combined conditions, ranges and lists.

2. Software / Tools Required: SQL-supported DBMS (such as MySQL), database client/SQL editor and a computer.

3. Theory: AND requires both conditions to be true. BETWEEN checks an inclusive range. IN checks whether a value matches any value in a given list.

4. Algorithm / Procedure:

• Open the SQL editor and connect to the database.

• Execute the setup statements, if provided, to create and populate the required tables.

• Run each task query in the given order.

• Verify the returned rows or the changes made to the table.

5. Setup / Initial Data:

```
CREATE TABLE player (name VARCHAR(30), team VARCHAR(20), runs INT);
INSERT INTO player (name, team, runs)
VALUES ('Asha', 'Eagles', 45), ('Meera', 'Tigers', 72),
('Arjun', 'Eagles', 30), ('Kiran', 'Lions', 88);
```

6. Program / Queries / Commands:

```
-- Task 1
SELECT name, runs FROM player
WHERE team = 'Eagles' AND runs > 35;

-- Task 2
SELECT name, runs FROM player
WHERE runs BETWEEN 40 AND 80;

-- Task 3
SELECT name, team FROM player
WHERE team IN ('Tigers', 'Lions');
```

7. Input: Four player records belonging to Eagles, Tigers and Lions.

8. Output:

name	runs
Asha	45
Meera	72

Task 1 returns Asha (45). Task 2 returns Asha (45) and Meera (72). Task 3 returns Meera (Tigers) and Kiran (Lions).

9. Result: The experiment was performed successfully. The required SQL operation was executed and the output was verified against the sample data.

10. Viva-Voce Questions with Answers:

Q. What does AND do in SQL?

Ans. It returns rows only when both connected conditions are true.

Q. Is BETWEEN inclusive?

Ans. Yes. BETWEEN includes both the lower and upper boundary values.

Q. What is the use of IN?

Ans. IN checks whether a value matches one of the values in a specified list.

EXPERIMENT 05

Pattern Matching with LIKE

1. Aim: Find book titles using the LIKE operator and wildcard patterns.

2. Software / Tools Required: SQL-supported DBMS (such as MySQL), database client/SQL editor and a computer.

3. Theory: LIKE performs pattern matching on text. The percent sign (%) represents zero or more characters, while underscore (_) represents exactly one character.

4. Algorithm / Procedure:

• Open the SQL editor and connect to the database.

• Execute the setup statements, if provided, to create and populate the required tables.

• Run each task query in the given order.

• Verify the returned rows or the changes made to the table.

5. Setup / Initial Data:

```
CREATE TABLE book (title VARCHAR(40), category VARCHAR(20), copies INT);
INSERT INTO book (title, category, copies)
VALUES ('Database Systems', 'Computer', 3),
('Digital Electronics', 'Electronics', 2),
('Data Structures in C', 'Computer', 2),
('Short Stories', 'Literature', 1);
```

6. Program / Queries / Commands:

```
-- Task 1: Starts with Data
SELECT title FROM book WHERE title LIKE 'Data%';

-- Task 2: Ends with C
SELECT title FROM book WHERE title LIKE '%C';

-- Task 3: Second letter is h
SELECT title FROM book WHERE title LIKE '_h%';
```

7. Input: Four book titles as listed in the setup.

8. Output:

Matching title (Task 1)
Database Systems
Data Structures in C

Task 2 returns “Data Structures in C”. Task 3 returns “Short Stories”.

9. Result: The experiment was performed successfully. The required SQL operation was executed and the output was verified against the sample data.

10. Viva-Voce Questions with Answers:

Q. What is LIKE used for?

Ans. LIKE is used to search text values that match a specified pattern.

Q. What does % mean in LIKE?

Ans. It matches zero or more characters.

Q. What does _ mean in LIKE?

Ans. It matches exactly one character.

EXPERIMENT 06

Sorting Rows and Removing Repeats

1. Aim: Sort product records, display distinct categories and retrieve the two most expensive products.

2. Software / Tools Required: SQL-supported DBMS (such as MySQL), database client/SQL editor and a computer.

3. Theory: ORDER BY sorts query results. DESC sorts from highest to lowest and ASC sorts from lowest to highest. DISTINCT removes duplicate values. LIMIT restricts the number of returned rows in MySQL and several other DBMSs.

4. Algorithm / Procedure:

• Open the SQL editor and connect to the database.

• Execute the setup statements, if provided, to create and populate the required tables.

• Run each task query in the given order.

• Verify the returned rows or the changes made to the table.

5. Setup / Initial Data:

```
CREATE TABLE product (product_name VARCHAR(30), category VARCHAR(20), price INT);
INSERT INTO product (product_name, category, price)
VALUES ('Pen', 'Stationery', 15), ('Notebook', 'Stationery', 60),
('Calculator', 'Electronics', 850), ('Bag', 'Office', 500);
```

6. Program / Queries / Commands:

```
-- Task 1
SELECT product_name, price FROM product ORDER BY price DESC;

-- Task 2
SELECT DISTINCT category FROM product ORDER BY category ASC;

-- Task 3
SELECT product_name, price FROM product
ORDER BY price DESC LIMIT 2;
```

7. Input: Pen (15), Notebook (60), Calculator (850), Bag (500).

8. Output:

product_name	price
Calculator	850
Bag	500

Task 2 returns Electronics, Office and Stationery. Task 3 returns Calculator (850) and Bag (500).

9. Result: The experiment was performed successfully. The required SQL operation was executed and the output was verified against the sample data.

10. Viva-Voce Questions with Answers:

Q. What is ORDER BY?

Ans. It sorts the rows returned by a query.

Q. What is DISTINCT?

Ans. DISTINCT removes duplicate values from the result.

Q. What does DESC mean?

Ans. DESC sorts values in descending order, from higher to lower.

EXPERIMENT 07

Using Aggregate Functions

1. Aim: Use COUNT, SUM and MAX aggregate functions on sales records.

2. Software / Tools Required: SQL-supported DBMS (such as MySQL), database client/SQL editor and a computer.

3. Theory: Aggregate functions perform calculations on a set of rows and return a single value. COUNT counts rows or non-NULL values, SUM totals numeric values and MAX returns the largest value.

4. Algorithm / Procedure:

• Open the SQL editor and connect to the database.

• Execute the setup statements, if provided, to create and populate the required tables.

• Run each task query in the given order.

• Verify the returned rows or the changes made to the table.

5. Setup / Initial Data:

```
CREATE TABLE sales (salesperson VARCHAR(20), region VARCHAR(20), amount INT);
INSERT INTO sales (salesperson, region, amount)
VALUES ('Deepak', 'North', 12000), ('Pooja', 'South', 8500),
('Deepak', 'South', 9500), ('Lata', 'North', 11000);
```

6. Program / Queries / Commands:

```
-- Task 1
SELECT COUNT(*) AS num_sales FROM sales;

-- Task 2
SELECT SUM(amount) AS total FROM sales;

-- Task 3
SELECT MAX(amount) AS highest FROM sales;
```

7. Input: Sales amounts: 12000, 8500, 9500 and 11000.

8. Output:

Aggregate result	Value
num_sales	4
total	41000
highest	12000

Task 1 returns num_sales = 4. Task 2 returns total = 41000. Task 3 returns highest = 12000.

9. Result: The experiment was performed successfully. The required SQL operation was executed and the output was verified against the sample data.

10. Viva-Voce Questions with Answers:

Q. What is an aggregate function?

Ans. It performs a calculation on multiple rows and returns a single result.

Q. What does COUNT(*) return?

Ans. It returns the total number of rows.

Q. What is the difference between SUM and MAX?

Ans. SUM adds numeric values; MAX returns the greatest value.

EXPERIMENT 08

Grouping Rows and Using HAVING

1. Aim: Group sales by region, calculate totals and filter groups using HAVING.

2. Software / Tools Required: SQL-supported DBMS (such as MySQL), database client/SQL editor and a computer.

3. Theory: GROUP BY combines rows that share the same value. Aggregate functions calculate a result for each group. HAVING filters groups after aggregation, whereas WHERE filters individual rows before grouping.

4. Algorithm / Procedure:

• Open the SQL editor and connect to the database.

• Execute the setup statements, if provided, to create and populate the required tables.

• Run each task query in the given order.

• Verify the returned rows or the changes made to the table.

5. Setup / Initial Data:

```
CREATE TABLE shop_sale (salesperson VARCHAR(20), region VARCHAR(20), amount INT);
INSERT INTO shop_sale (salesperson, region, amount)
VALUES ('Deepak', 'North', 12000), ('Pooja', 'South', 8500),
('Deepak', 'South', 9500), ('Lata', 'North', 11000);
```

6. Program / Queries / Commands:

```
-- Task 1
SELECT region, COUNT(*) AS num_sales
FROM shop_sale GROUP BY region ORDER BY region;

-- Task 2
SELECT region, SUM(amount) AS total
FROM shop_sale GROUP BY region ORDER BY region;

-- Task 3
SELECT region, SUM(amount) AS total
FROM shop_sale GROUP BY region
HAVING SUM(amount) > 20000 ORDER BY region;
```

7. Input: North region amounts: 12000 and 11000; South region amounts: 8500 and 9500.

8. Output:

region	total
North	23000

Task 1 returns North = 2 and South = 2. Task 2 returns North = 23000 and South = 18000. Task 3 returns North = 23000.

9. Result: The experiment was performed successfully. The required SQL operation was executed and the output was verified against the sample data.

10. Viva-Voce Questions with Answers:

Q. Why is GROUP BY used?

Ans. It groups rows with the same value so aggregate calculations can be performed per group.

Q. What is HAVING?

Ans. HAVING filters grouped results, usually using aggregate conditions.

Q. How is HAVING different from WHERE?

Ans. WHERE filters rows before grouping; HAVING filters groups after grouping.

EXPERIMENT 09

Aliases and CASE

1. Aim: Rename output columns with AS and use CASE to display a result based on marks.

2. Software / Tools Required: SQL-supported DBMS (such as MySQL), database client/SQL editor and a computer.

3. Theory: An alias gives a temporary name to a column or expression in query output. CASE evaluates conditions and returns a value for the first matching condition.

4. Algorithm / Procedure:

• Open the SQL editor and connect to the database.

• Execute the setup statements, if provided, to create and populate the required tables.

• Run each task query in the given order.

• Verify the returned rows or the changes made to the table.

5. Setup / Initial Data:

```
CREATE TABLE exam_result (name VARCHAR(20), marks INT);
INSERT INTO exam_result (name, marks)
VALUES ('Asha', 92), ('Rohan', 47), ('Neha', 68), ('Meera', 33);
```

6. Program / Queries / Commands:

```
-- Task 1
SELECT name AS student_name, marks AS score FROM exam_result;

-- Task 2
SELECT name,
CASE WHEN marks >= 40 THEN 'Pass' ELSE 'Fail' END AS result
FROM exam_result;

-- Task 3
SELECT name, marks + 5 AS with_bonus FROM exam_result;
```

7. Input: Asha (92), Rohan (47), Neha (68), Meera (33).

8. Output:

name	result
Asha	Pass
Rohan	Pass
Neha	Pass
Meera	Fail

Task 1 returns columns student_name and score. Task 3 returns Asha 97, Rohan 52, Neha 73 and Meera 38.

9. Result: The experiment was performed successfully. The required SQL operation was executed and the output was verified against the sample data.

10. Viva-Voce Questions with Answers:

Q. What is an alias in SQL?

Ans. An alias is a temporary alternative name for a column or table.

Q. What is the purpose of AS?

Ans. AS assigns an alias to a selected column or expression.

Q. What is CASE used for?

Ans. CASE returns a value based on one or more conditions.

EXPERIMENT 10

Joining Two Tables with INNER JOIN

1. Aim: Join department and employee tables using INNER JOIN on dept_id.

2. Software / Tools Required: SQL-supported DBMS (such as MySQL), database client/SQL editor and a computer.

3. Theory: INNER JOIN returns rows where the join condition matches in both tables. A common key such as dept_id connects related records.

4. Algorithm / Procedure:

• Open the SQL editor and connect to the database.

• Execute the setup statements, if provided, to create and populate the required tables.

• Run each task query in the given order.

• Verify the returned rows or the changes made to the table.

5. Setup / Initial Data:

```
CREATE TABLE department (dept_id INT, dept_name VARCHAR(20));
INSERT INTO department (dept_id, dept_name)
VALUES (10, 'Accounts'), (20, 'Sales'), (40, 'Research');

CREATE TABLE employee (emp_name VARCHAR(30), dept_id INT, salary INT);
INSERT INTO employee (emp_name, dept_id, salary)
VALUES ('Anita', 10, 90000), ('Lata', 20, 55000),
('Deepak', 10, 60000), ('Pooja', 20, 42000);
```

6. Program / Queries / Commands:

```
-- Task 1
SELECT e.emp_name, d.dept_name
FROM employee e INNER JOIN department d
ON e.dept_id = d.dept_id
ORDER BY e.emp_name;

-- Task 2
SELECT e.emp_name
FROM employee e INNER JOIN department d
ON e.dept_id = d.dept_id
WHERE d.dept_name = 'Sales'
ORDER BY e.emp_name;

-- Task 3
SELECT e.emp_name, d.dept_name, e.salary
FROM employee e INNER JOIN department d
ON e.dept_id = d.dept_id
ORDER BY e.salary DESC;
```

7. Input: Departments: Accounts (10), Sales (20), Research (40); four employees linked to department IDs 10 and 20.

8. Output:

emp_name	dept_name	salary
Anita	Accounts	90000
Deepak	Accounts	60000
Lata	Sales	55000
Pooja	Sales	42000

Task 1 returns Anita–Accounts, Deepak–Accounts, Lata–Sales and Pooja–Sales. Task 2 returns Lata and Pooja. Task 3 is shown above.

9. Result: The experiment was performed successfully. The required SQL operation was executed and the output was verified against the sample data.

10. Viva-Voce Questions with Answers:

Q. What is a JOIN?

Ans. JOIN combines related rows from two or more tables.

Q. What does INNER JOIN return?

Ans. It returns only rows that satisfy the join condition in both tables.

Q. Why is dept_id used here?

Ans. It is the common column that relates each employee to a department.

EXPERIMENT 11

Other Kinds of Joins

1. Aim: Use LEFT JOIN, CROSS JOIN and NATURAL JOIN with team and player tables.

2. Software / Tools Required: SQL-supported DBMS (such as MySQL), database client/SQL editor and a computer.

3. Theory: LEFT JOIN keeps every row from the left table and includes matching right-table rows. CROSS JOIN produces every possible pair. NATURAL JOIN automatically joins tables using columns with the same names.

4. Algorithm / Procedure:

• Open the SQL editor and connect to the database.

• Execute the setup statements, if provided, to create and populate the required tables.

• Run each task query in the given order.

• Verify the returned rows or the changes made to the table.

5. Setup / Initial Data:

```
CREATE TABLE team (team_id INT, team_name VARCHAR(20));
INSERT INTO team (team_id, team_name)
VALUES (1, 'Eagles'), (2, 'Tigers'), (3, 'Lions');

CREATE TABLE player (player_name VARCHAR(30), team_id INT, age INT);
INSERT INTO player (player_name, team_id, age)
VALUES ('Asha', 1, 24), ('Meera', 1, 22), ('Arjun', 2, 27);
```

6. Program / Queries / Commands:

```
-- Task 1: Include teams with no players
SELECT t.team_name, p.player_name
FROM team t LEFT JOIN player p ON t.team_id = p.team_id
ORDER BY t.team_name, p.player_name;

-- Task 2: Every possible team/player pair
SELECT t.team_name, p.player_name
FROM team t CROSS JOIN player p
ORDER BY t.team_name, p.player_name;

-- Task 3: Match automatically on the common team_id column
SELECT player_name, team_name
FROM player NATURAL JOIN team
ORDER BY player_name;
```

7. Input: Teams: Eagles, Tigers, Lions. Players: Asha and Meera in team 1; Arjun in team 2.

8. Output:

team_name	player_name
Eagles	Asha
Eagles	Meera
Lions	NULL
Tigers	Arjun

Task 2 returns all 9 team/player combinations. Task 3 returns Arjun–Tigers, Asha–Eagles and Meera–Eagles.

9. Result: The experiment was performed successfully. The required SQL operation was executed and the output was verified against the sample data.

10. Viva-Voce Questions with Answers:

Q. What does LEFT JOIN do?

Ans. It returns all rows from the left table and matching rows from the right table; unmatched right-side values are NULL.

Q. What is CROSS JOIN?

Ans. It returns the Cartesian product: every row of one table paired with every row of the other.

Q. What is NATURAL JOIN?

Ans. It joins tables automatically using columns that have identical names.

EXPERIMENT 12

Using a Subquery

1. Aim: Use a subquery with IN to find students who are members of both clubs.

2. Software / Tools Required: SQL-supported DBMS (such as MySQL), database client/SQL editor and a computer.

3. Theory: A subquery is a query nested inside another query. IN checks whether a value from the outer query appears in the list returned by the subquery.

4. Algorithm / Procedure:

• Open the SQL editor and connect to the database.

• Execute the setup statements, if provided, to create and populate the required tables.

• Run each task query in the given order.

• Verify the returned rows or the changes made to the table.

5. Setup / Initial Data:

```
CREATE TABLE club_a (student_name VARCHAR(30));
INSERT INTO club_a (student_name) VALUES ('Asha'), ('Rohan'), ('Neha');
CREATE TABLE club_b (student_name VARCHAR(30));
INSERT INTO club_b (student_name) VALUES ('Neha'), ('Pooja'), ('Meera');
```

6. Program / Queries / Commands:

```
-- Task 1
SELECT student_name FROM club_a
WHERE student_name IN (SELECT student_name FROM club_b);

-- Task 2
SELECT COUNT(*) AS members FROM club_b;

-- Task 3
SELECT student_name FROM club_a ORDER BY student_name;
```

7. Input: Club A: Asha, Rohan, Neha. Club B: Neha, Pooja, Meera.

8. Output:

student_name
Neha

Task 2 returns Neha. Task 3 returns Asha, Neha and Rohan.

9. Result: The experiment was performed successfully. The required SQL operation was executed and the output was verified against the sample data.

10. Viva-Voce Questions with Answers:

Q. What is a subquery?

Ans. A subquery is a query written inside another SQL query.

Q. What does IN do?

Ans. IN checks whether a value matches any value in a specified list or subquery result.

Q. Can a subquery be used with SELECT?

Ans. Yes. A subquery can be used in places such as WHERE, FROM and SELECT, depending on the query.

EXPERIMENT 13

UNION and INTERSECT

1. Aim: Combine club member lists with UNION and UNION ALL, and find common names with INTERSECT.

2. Software / Tools Required: SQL-supported DBMS (such as MySQL), database client/SQL editor and a computer.

3. Theory: UNION combines result sets and removes duplicates. UNION ALL keeps duplicates. INTERSECT returns rows common to both result sets. The SELECT statements must return compatible columns.

4. Algorithm / Procedure:

• Open the SQL editor and connect to the database.

• Execute the setup statements, if provided, to create and populate the required tables.

• Run each task query in the given order.

• Verify the returned rows or the changes made to the table.

5. Setup / Initial Data:

```
CREATE TABLE club_a (student_name VARCHAR(30));
INSERT INTO club_a (student_name) VALUES ('Asha'), ('Rohan'), ('Neha');
CREATE TABLE club_b (student_name VARCHAR(30));
INSERT INTO club_b (student_name) VALUES ('Neha'), ('Pooja'), ('Meera');
```

6. Program / Queries / Commands:

```
-- Task 1: Unique names from both clubs
SELECT student_name FROM club_a
UNION
SELECT student_name FROM club_b
ORDER BY student_name;

-- Task 2: Names present in both clubs
SELECT student_name FROM club_a
INTERSECT
SELECT student_name FROM club_b
ORDER BY student_name;

-- Task 3: Keep repeated names
SELECT student_name FROM club_a
UNION ALL
SELECT student_name FROM club_b
ORDER BY student_name;
```

7. Input: Club A: Asha, Rohan, Neha. Club B: Neha, Pooja, Meera.

8. Output:

student_name (UNION)
Asha
Meera
Neha
Pooja
Rohan

Task 2 (INTERSECT) returns Neha. Task 3 (UNION ALL) returns Asha, Meera, Neha, Neha, Pooja and Rohan.

9. Result: The experiment was performed successfully. The required SQL operation was executed and the output was verified against the sample data.

10. Viva-Voce Questions with Answers:

Q. What does UNION do?

Ans. It combines rows from two compatible SELECT results and removes duplicates.

Q. How is UNION ALL different from UNION?

Ans. UNION ALL retains duplicate rows, while UNION removes duplicates.

Q. What does INTERSECT return?

Ans. It returns rows that appear in both query results. Support depends on the DBMS/version.

EXPERIMENT 14

Creating and Using a View

1. Aim: Create a view named high_scores and retrieve all rows and selected names from it.

2. Software / Tools Required: SQL-supported DBMS (such as MySQL), database client/SQL editor and a computer.

3. Theory: A view is a named query that behaves like a virtual table. It can simplify repeated queries and present selected data from one or more tables.

4. Algorithm / Procedure:

• Open the SQL editor and connect to the database.

• Execute the setup statements, if provided, to create and populate the required tables.

• Run each task query in the given order.

• Verify the returned rows or the changes made to the table.

5. Setup / Initial Data:

```
CREATE TABLE score (name VARCHAR(30), subject VARCHAR(20), marks INT);
INSERT INTO score (name, subject, marks)
VALUES ('Asha', 'Maths', 78), ('Rohan', 'Maths', 64),
('Neha', 'Maths', 88), ('Kiran', 'Maths', 52);
```

6. Program / Queries / Commands:

```
-- Task 1
CREATE VIEW high_scores AS
SELECT name, marks FROM score WHERE marks >= 70;

-- Task 2
SELECT * FROM high_scores;

-- Task 3
SELECT name FROM high_scores ORDER BY name;
```

7. Input: Maths scores: Asha 78, Rohan 64, Neha 88, Kiran 52.

8. Output:

name	marks
Asha	78
Neha	88

Task 3 returns the names Asha and Neha, in alphabetical order.

9. Result: The experiment was performed successfully. The required SQL operation was executed and the output was verified against the sample data.

10. Viva-Voce Questions with Answers:

Q. What is a view?

Ans. A view is a virtual table based on a stored SQL query.

Q. Does a view usually store its own copy of the data?

Ans. A standard view stores the query definition; its results are produced from the underlying tables.

Q. Why are views useful?

Ans. They simplify complex queries and can limit which columns or rows users see.

EXPERIMENT 15

Indexes and Adding a Row

1. Aim: Create an index on the city column, find Delhi customers and insert a new customer.

2. Software / Tools Required: SQL-supported DBMS (such as MySQL), database client/SQL editor and a computer.

3. Theory: An index is a database structure that can speed up searches on indexed columns. It may use extra storage and can add overhead to INSERT, UPDATE and DELETE operations.

4. Algorithm / Procedure:

• Open the SQL editor and connect to the database.

• Execute the setup statements, if provided, to create and populate the required tables.

• Run each task query in the given order.

• Verify the returned rows or the changes made to the table.

5. Setup / Initial Data:

```
CREATE TABLE customer (
customer_id INT PRIMARY KEY,
name VARCHAR(30) NOT NULL,
city VARCHAR(20)
);
INSERT INTO customer (customer_id, name, city)
VALUES (1, 'Kiran', 'Kochi'), (2, 'Deepak', 'Delhi'),
(3, 'Lata', 'Mumbai'), (4, 'Asha', 'Delhi');
```

6. Program / Queries / Commands:

```
-- Task 1
CREATE INDEX idx_city ON customer(city);

-- Task 2
SELECT name FROM customer
WHERE city = 'Delhi' ORDER BY name;

-- Task 3
INSERT INTO customer (customer_id, name, city)
VALUES (5, 'Arjun', 'Pune');
SELECT * FROM customer ORDER BY customer_id;
```

7. Input: Initial customers: Kiran/Kochi, Deepak/Delhi, Lata/Mumbai, Asha/Delhi. New row: Arjun/Pune.

8. Output:

customer_id	name	city
1	Kiran	Kochi
2	Deepak	Delhi
3	Lata	Mumbai
4	Asha	Delhi
5	Arjun	Pune

Task 2 returns Asha and Deepak. Task 3 displays the five customers shown above.

9. Result: The experiment was performed successfully. The required SQL operation was executed and the output was verified against the sample data.

10. Viva-Voce Questions with Answers:

Q. What is an index?

Ans. An index is a database structure that helps find rows more efficiently.

Q. What is the purpose of CREATE INDEX?

Ans. It creates an index on one or more table columns.

Q. Can an index slow down data modification?

Ans. Yes. The DBMS must maintain the index when rows are inserted, updated or deleted.

EXPERIMENT 01 – SQL EXECUTION RESULTS

Creating a Table and Adding Rows

Executed using SQLite 3.46.1 | Fresh in-memory database for this experiment

NO SEPARATE SET-UP REQUIRED

Task 1 – Create the student table

```
sqlite> CREATE TABLE student (roll_no INT, name VARCHAR(30), marks INT);
Query executed successfully. Rows affected: 0.
Result: No data rows returned (DDL/DML statement).
```

Task 2 – Insert three student records

```
sqlite> INSERT INTO student (roll_no, name, marks) VALUES (101, 'Asha', 78), (102, 'Rohan', 64), (103,
    'Neha', 88);
Query executed successfully. Rows affected: 3.
Result: No data rows returned (DDL/DML statement).
```

Task 3 – Display all rows

```
sqlite> SELECT * FROM student;
Query executed successfully.
Result:
roll_no | name   | marks
--------+--------+-------
101     | Asha   | 78
102     | Rohan  | 64
103     | Neha   | 88
```

EXPERIMENT 02 – SQL EXECUTION RESULTS

Updating and Deleting Rows

Executed using SQLite 3.46.1 | Fresh in-memory database for this experiment

SET-UP SQL EXECUTED

```
CREATE TABLE stock (item_code INT, item_name VARCHAR(30), quantity INT);
INSERT INTO stock (item_code, item_name, quantity) VALUES (1, 'Notebook', 50), (2, 'Pen', 120), (3,
    'Bag', 0);
```

Task 1 – Set Notebook quantity to 80

```
sqlite> UPDATE stock SET quantity = 80 WHERE item_code = 1;
sqlite> SELECT * FROM stock;
Query executed successfully.
Result:
item_code | item_name | quantity
----------+-----------+---------
1         | Notebook  | 80
2         | Pen       | 120
3         | Bag       | 0
```

Task 2 – Add 10 to Pen quantity

```
sqlite> UPDATE stock SET quantity = quantity + 10 WHERE item_code = 2;
sqlite> SELECT * FROM stock;
Query executed successfully.
Result:
item_code | item_name | quantity
----------+-----------+---------
1         | Notebook  | 80
2         | Pen       | 130
3         | Bag       | 0
```

Task 3 – Delete item_code 3

```
sqlite> DELETE FROM stock WHERE item_code = 3;
sqlite> SELECT * FROM stock;
Query executed successfully.
Result:
item_code | item_name | quantity
----------+-----------+---------
1         | Notebook  | 80
2         | Pen       | 130
```

EXPERIMENT 03 – SQL EXECUTION RESULTS

Selecting Columns and Rows with WHERE

Executed using SQLite 3.46.1 | Fresh in-memory database for this experiment

SET-UP SQL EXECUTED

```
CREATE TABLE student (roll_no INT, name VARCHAR(30), marks INT);
INSERT INTO student (roll_no, name, marks) VALUES (101, 'Asha', 78), (102, 'Rohan', 64), (103, 'Neha',
    88), (104, 'Deepak', 52);
```

Task 1 – Display name and marks

```
sqlite> SELECT name, marks FROM student;
Query executed successfully.
Result:
name   | marks
-------+-------
Asha   | 78
Rohan  | 64
Neha   | 88
Deepak | 52
```

Task 2 – Students with marks greater than 60

```
sqlite> SELECT * FROM student WHERE marks > 60;
Query executed successfully.
Result:
roll_no | name   | marks
--------+--------+-------
101     | Asha   | 78
102     | Rohan  | 64
103     | Neha   | 88
```

Task 3 – Student with roll number 103

```
sqlite> SELECT name FROM student WHERE roll_no = 103;
Query executed successfully.
Result:
name
------
Neha
```

EXPERIMENT 04 – SQL EXECUTION RESULTS

Combining Conditions

Executed using SQLite 3.46.1 | Fresh in-memory database for this experiment

SET-UP SQL EXECUTED

```
CREATE TABLE player (name VARCHAR(30), team VARCHAR(20), runs INT);
INSERT INTO player (name, team, runs) VALUES ('Asha', 'Eagles', 45), ('Meera', 'Tigers', 72), ('Arjun',
    'Eagles', 30), ('Kiran', 'Lions', 88);
```

Task 1 – Eagles players scoring over 35

```
sqlite> SELECT name, runs FROM player WHERE team = 'Eagles' AND runs > 35;
Query executed successfully.
Result:
name   | runs
-------+-------
Asha   | 45
```

Task 2 – Runs between 40 and 80

```
sqlite> SELECT name, runs FROM player WHERE runs BETWEEN 40 AND 80;
Query executed successfully.
Result:
name   | runs
-------+-------
Asha   | 45
Meera  | 72
```

Task 3 – Players in Tigers or Lions

```
sqlite> SELECT name, team FROM player WHERE team IN ('Tigers', 'Lions');
Query executed successfully.
Result:
name   | team
-------+-------
Meera  | Tigers
Kiran  | Lions
```

EXPERIMENT 05 – SQL EXECUTION RESULTS

Pattern Matching with LIKE

Executed using SQLite 3.46.1 | Fresh in-memory database for this experiment

SET-UP SQL EXECUTED

```
CREATE TABLE book (title VARCHAR(40), category VARCHAR(20), copies INT);
INSERT INTO book (title, category, copies) VALUES ('Database Systems', 'Computer', 3), ('Digital
    Electronics', 'Electronics', 2), ('Data Structures in C', 'Computer', 2), ('Short Stories',
    'Literature', 1);
```

Task 1 – Titles starting with Data

```
sqlite> SELECT title FROM book WHERE title LIKE 'Data%';
Query executed successfully.
Result:
title
--------------------
Database Systems
Data Structures in C
```

Task 2 – Titles ending with C

```
sqlite> SELECT title FROM book WHERE title LIKE '%C';
Query executed successfully.
Result:
title
--------------------
Data Structures in C
```

Task 3 – Titles with h as second letter

```
sqlite> SELECT title FROM book WHERE title LIKE '_h%';
Query executed successfully.
Result:
title
-------------
Short Stories
```

EXPERIMENT 06 – SQL EXECUTION RESULTS

Sorting Rows and Removing Repeats

Executed using SQLite 3.46.1 | Fresh in-memory database for this experiment

SET-UP SQL EXECUTED

```
CREATE TABLE product (product_name VARCHAR(30), category VARCHAR(20), price INT);
INSERT INTO product (product_name, category, price) VALUES ('Pen', 'Stationery', 15), ('Notebook',
    'Stationery', 60), ('Calculator', 'Electronics', 850), ('Bag', 'Office', 500);
```

Task 1 – Products sorted by descending price

```
sqlite> SELECT product_name, price FROM product ORDER BY price DESC;
Query executed successfully.
Result:
product_name | price
-------------+-------
Calculator   | 850
Bag          | 500
Notebook     | 60
Pen          | 15
```

Task 2 – Unique categories sorted by name

```
sqlite> SELECT DISTINCT category FROM product ORDER BY category;
Query executed successfully.
Result:
category
-----------
Electronics
Office
Stationery
```

Task 3 – Two most expensive products

```
sqlite> SELECT product_name, price FROM product ORDER BY price DESC LIMIT 2;
Query executed successfully.
Result:
product_name | price
-------------+-------
Calculator   | 850
Bag          | 500
```

EXPERIMENT 07 – SQL EXECUTION RESULTS

Using Aggregate Functions

Executed using SQLite 3.46.1 | Fresh in-memory database for this experiment

SET-UP SQL EXECUTED

```
CREATE TABLE sales (salesperson VARCHAR(20), region VARCHAR(20), amount INT);
INSERT INTO sales (salesperson, region, amount) VALUES ('Deepak', 'North', 12000), ('Pooja', 'South',
    8500), ('Deepak', 'South', 9500), ('Lata', 'North', 11000);
```

Task 1 – Count sales rows

```
sqlite> SELECT COUNT(*) AS num_sales FROM sales;
Query executed successfully.
Result:
num_sales
---------
4
```

Task 2 – Total sales amount

```
sqlite> SELECT SUM(amount) AS total FROM sales;
Query executed successfully.
Result:
total
------
41000
```

Task 3 – Highest sales amount

```
sqlite> SELECT MAX(amount) AS highest FROM sales;
Query executed successfully.
Result:
highest
-------
12000
```

EXPERIMENT 08 – SQL EXECUTION RESULTS

Grouping Rows and Using HAVING

Executed using SQLite 3.46.1 | Fresh in-memory database for this experiment

SET-UP SQL EXECUTED

```
CREATE TABLE shop_sale (salesperson VARCHAR(20), region VARCHAR(20), amount INT);
INSERT INTO shop_sale (salesperson, region, amount) VALUES ('Deepak', 'North', 12000), ('Pooja',
    'South', 8500), ('Deepak', 'South', 9500), ('Lata', 'North', 11000);
```

Task 1 – Count sales in each region

```
sqlite> SELECT region, COUNT(*) AS num_sales FROM shop_sale GROUP BY region ORDER BY region;
Query executed successfully.
Result:
region | num_sales
-------+----------
North  | 2
South  | 2
```

Task 2 – Total amount in each region

```
sqlite> SELECT region, SUM(amount) AS total FROM shop_sale GROUP BY region ORDER BY region;
Query executed successfully.
Result:
region | total
-------+-------
North  | 23000
South  | 18000
```

Task 3 – Regions with total over 20000

```
sqlite> SELECT region, SUM(amount) AS total FROM shop_sale GROUP BY region HAVING SUM(amount) > 20000
    ORDER BY region;
Query executed successfully.
Result:
region | total
-------+-------
North  | 23000
```

EXPERIMENT 09 – SQL EXECUTION RESULTS

Aliases and CASE

Executed using SQLite 3.46.1 | Fresh in-memory database for this experiment

SET-UP SQL EXECUTED

```
CREATE TABLE exam_result (name VARCHAR(20), marks INT);
INSERT INTO exam_result (name, marks) VALUES ('Asha', 92), ('Rohan', 47), ('Neha', 68), ('Meera', 33);
```

Task 1 – Rename columns with AS

```
sqlite> SELECT name AS student_name, marks AS score FROM exam_result;
Query executed successfully.
Result:
student_name | score
-------------+-------
Asha         | 92
Rohan        | 47
Neha         | 68
Meera        | 33
```

Task 2 – Show Pass or Fail using CASE

```
sqlite> SELECT name, CASE WHEN marks >= 40 THEN 'Pass' ELSE 'Fail' END AS result FROM exam_result;
Query executed successfully.
Result:
name   | result
-------+-------
Asha   | Pass
Rohan  | Pass
Neha   | Pass
Meera  | Fail
```

Task 3 – Add five bonus marks

```
sqlite> SELECT name, marks + 5 AS with_bonus FROM exam_result;
Query executed successfully.
Result:
name   | with_bonus
-------+-----------
Asha   | 97
Rohan  | 52
Neha   | 73
Meera  | 38
```

EXPERIMENT 10 – SQL EXECUTION RESULTS

Joining Two Tables with INNER JOIN

Executed using SQLite 3.46.1 | Fresh in-memory database for this experiment

SET-UP SQL EXECUTED

```
CREATE TABLE department (dept_id INT, dept_name VARCHAR(20));
INSERT INTO department (dept_id, dept_name) VALUES (10, 'Accounts'), (20, 'Sales'), (40, 'Research');
CREATE TABLE employee (emp_name VARCHAR(30), dept_id INT, salary INT);
INSERT INTO employee (emp_name, dept_id, salary) VALUES ('Anita', 10, 90000), ('Lata', 20, 55000),
    ('Deepak', 10, 60000), ('Pooja', 20, 42000);
```

Task 1 – Employee and department names

```
sqlite> SELECT e.emp_name, d.dept_name FROM employee e INNER JOIN department d ON e.dept_id = d.dept_id
    ORDER BY e.emp_name;
Query executed successfully.
Result:
emp_name | dept_name
---------+----------
Anita    | Accounts
Deepak   | Accounts
Lata     | Sales
Pooja    | Sales
```

Task 2 – Employees in Sales department

```
sqlite> SELECT e.emp_name FROM employee e INNER JOIN department d ON e.dept_id = d.dept_id WHERE
    d.dept_name = 'Sales' ORDER BY e.emp_name;
Query executed successfully.
Result:
emp_name
--------
Lata
Pooja
```

Task 3 – Employee, department and salary

```
sqlite> SELECT e.emp_name, d.dept_name, e.salary FROM employee e INNER JOIN department d ON e.dept_id =
    d.dept_id ORDER BY e.salary DESC;
Query executed successfully.
Result:
emp_name | dept_name | salary
---------+-----------+-------
Anita    | Accounts  | 90000
Deepak   | Accounts  | 60000
Lata     | Sales     | 55000
Pooja    | Sales     | 42000
```

EXPERIMENT 11 – SQL EXECUTION RESULTS

Other Kinds of Joins

Executed using SQLite 3.46.1 | Fresh in-memory database for this experiment

SET-UP SQL EXECUTED

```
CREATE TABLE team (team_id INT, team_name VARCHAR(20));
INSERT INTO team (team_id, team_name) VALUES (1, 'Eagles'), (2, 'Tigers'), (3, 'Lions');
CREATE TABLE player (player_name VARCHAR(30), team_id INT, age INT);
INSERT INTO player (player_name, team_id, age) VALUES ('Asha', 1, 24), ('Meera', 1, 22), ('Arjun', 2,
    27);
```

Task 1 – LEFT JOIN including empty team

```
sqlite> SELECT t.team_name, p.player_name FROM team t LEFT JOIN player p ON t.team_id = p.team_id ORDER
    BY t.team_name, p.player_name;
Query executed successfully.
Result:
team_name | player_name
----------+------------
Eagles    | Asha
Eagles    | Meera
Lions     | NULL
Tigers    | Arjun
```

Task 2 – Every team/player pair using CROSS JOIN

```
sqlite> SELECT t.team_name, p.player_name FROM team t CROSS JOIN player p ORDER BY t.team_name,
    p.player_name;
Query executed successfully.
Result:
team_name | player_name
----------+------------
Eagles    | Arjun
Eagles    | Asha
Eagles    | Meera
Lions     | Arjun
Lions     | Asha
Lions     | Meera
Tigers    | Arjun
Tigers    | Asha
Tigers    | Meera
```

Task 3 – Player and team using NATURAL JOIN

```
sqlite> SELECT player_name, team_name FROM player NATURAL JOIN team ORDER BY player_name;
Query executed successfully.
Result:
player_name | team_name
------------+----------
Arjun       | Tigers
Asha        | Eagles
Meera       | Eagles
```

EXPERIMENT 12 – SQL EXECUTION RESULTS

Using a Subquery

Executed using SQLite 3.46.1 | Fresh in-memory database for this experiment

SET-UP SQL EXECUTED

```
CREATE TABLE club_a (student_name VARCHAR(30));
INSERT INTO club_a (student_name) VALUES ('Asha'), ('Rohan'), ('Neha');
CREATE TABLE club_b (student_name VARCHAR(30));
INSERT INTO club_b (student_name) VALUES ('Neha'), ('Pooja'), ('Meera');
```

Task 1 – Names common to both clubs

```
sqlite> SELECT student_name FROM club_a WHERE student_name IN (SELECT student_name FROM club_b);
Query executed successfully.
Result:
student_name
------------
Neha
```

Task 2 – Count names in club_b

```
sqlite> SELECT COUNT(*) AS members FROM club_b;
Query executed successfully.
Result:
members
-------
3
```

Task 3 – Names in club_a sorted alphabetically

```
sqlite> SELECT student_name FROM club_a ORDER BY student_name;
Query executed successfully.
Result:
student_name
------------
Asha
Neha
Rohan
```

EXPERIMENT 13 – SQL EXECUTION RESULTS

UNION and INTERSECT

Executed using SQLite 3.46.1 | Fresh in-memory database for this experiment

SET-UP SQL EXECUTED

```
CREATE TABLE club_a (student_name VARCHAR(30));
INSERT INTO club_a (student_name) VALUES ('Asha'), ('Rohan'), ('Neha');
CREATE TABLE club_b (student_name VARCHAR(30));
INSERT INTO club_b (student_name) VALUES ('Neha'), ('Pooja'), ('Meera');
```

Task 1 – Unique names using UNION

```
sqlite> SELECT student_name FROM club_a UNION SELECT student_name FROM club_b ORDER BY student_name;
Query executed successfully.
Result:
student_name
------------
Asha
Meera
Neha
Pooja
Rohan
```

Task 2 – Common names using INTERSECT

```
sqlite> SELECT student_name FROM club_a INTERSECT SELECT student_name FROM club_b ORDER BY student_name;
Query executed successfully.
Result:
student_name
------------
Neha
```

Task 3 – All names including repeats using UNION ALL

```
sqlite> SELECT student_name FROM club_a UNION ALL SELECT student_name FROM club_b ORDER BY student_name;
Query executed successfully.
Result:
student_name
------------
Asha
Meera
Neha
Neha
Pooja
Rohan
```

EXPERIMENT 14 – SQL EXECUTION RESULTS

Creating and Using a View

Executed using SQLite 3.46.1 | Fresh in-memory database for this experiment

SET-UP SQL EXECUTED

```
CREATE TABLE score (name VARCHAR(30), subject VARCHAR(20), marks INT);
INSERT INTO score (name, subject, marks) VALUES ('Asha', 'Maths', 78), ('Rohan', 'Maths', 64), ('Neha',
    'Maths', 88), ('Kiran', 'Maths', 52);
```

Task 1 – Create high_scores view

```
sqlite> CREATE VIEW high_scores AS SELECT name, marks FROM score WHERE marks >= 70;
Query executed successfully. Rows affected: 0.
Result: No data rows returned (DDL/DML statement).
```

Task 2 – Display all rows in high_scores

```
sqlite> SELECT * FROM high_scores;
Query executed successfully.
Result:
name   | marks
-------+-------
Asha   | 78
Neha   | 88
```

Task 3 – Display names in high_scores sorted

```
sqlite> SELECT name FROM high_scores ORDER BY name;
Query executed successfully.
Result:
name
------
Asha
Neha
```

EXPERIMENT 15 – SQL EXECUTION RESULTS

Indexes and Adding a Row

Executed using SQLite 3.46.1 | Fresh in-memory database for this experiment

SET-UP SQL EXECUTED

```
CREATE TABLE customer (customer_id INT PRIMARY KEY, name VARCHAR(30) NOT NULL, city VARCHAR(20));
INSERT INTO customer (customer_id, name, city) VALUES (1, 'Kiran', 'Kochi'), (2, 'Deepak', 'Delhi'),
    (3, 'Lata', 'Mumbai'), (4, 'Asha', 'Delhi');
```

Task 1 – Create index on city

```
sqlite> CREATE INDEX idx_city ON customer(city);
Query executed successfully. Rows affected: 0.
Result: No data rows returned (DDL/DML statement).
```

Task 2 – Customers living in Delhi

```
sqlite> SELECT name FROM customer WHERE city = 'Delhi' ORDER BY name;
Query executed successfully.
Result:
name
------
Asha
Deepak
```

Task 3 – Insert Arjun and display customers

```
sqlite> INSERT INTO customer (customer_id, name, city) VALUES (5, 'Arjun', 'Pune');
sqlite> SELECT * FROM customer ORDER BY customer_id;
Query executed successfully.
Result:
customer_id | name   | city
------------+--------+-------
1           | Kiran  | Kochi
2           | Deepak | Delhi
3           | Lata   | Mumbai
4           | Asha   | Delhi
5           | Arjun  | Pune
```